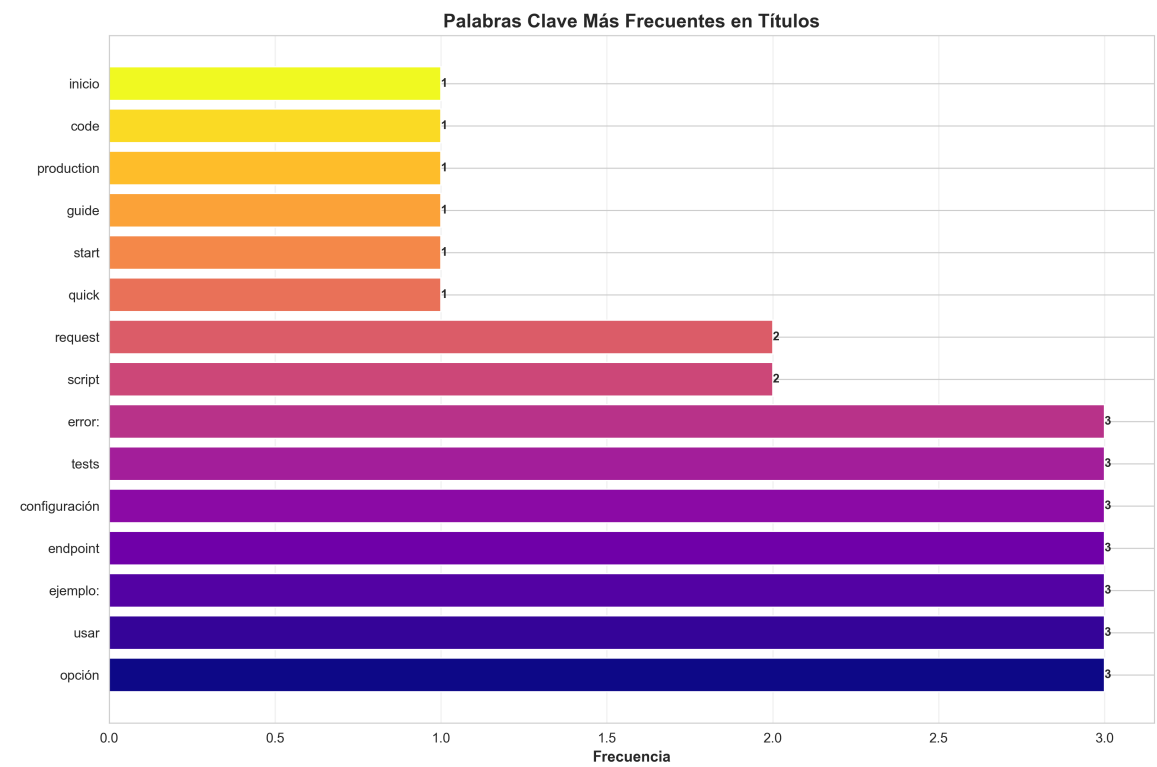
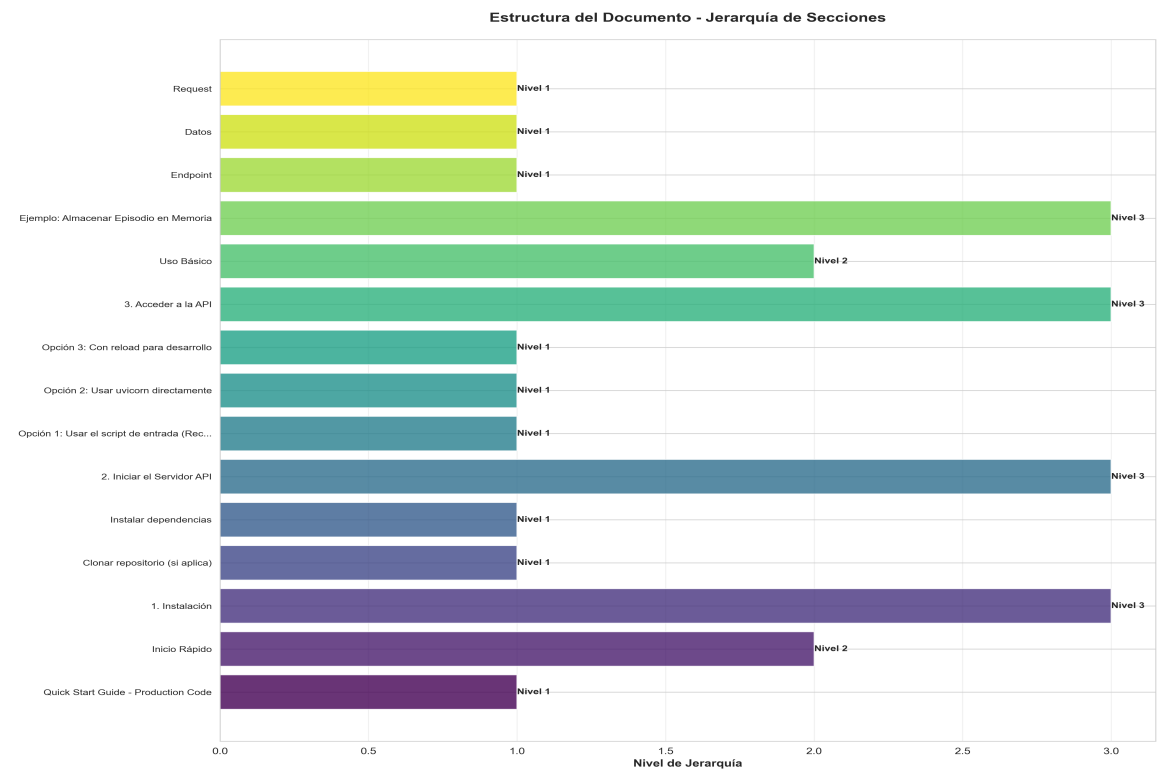
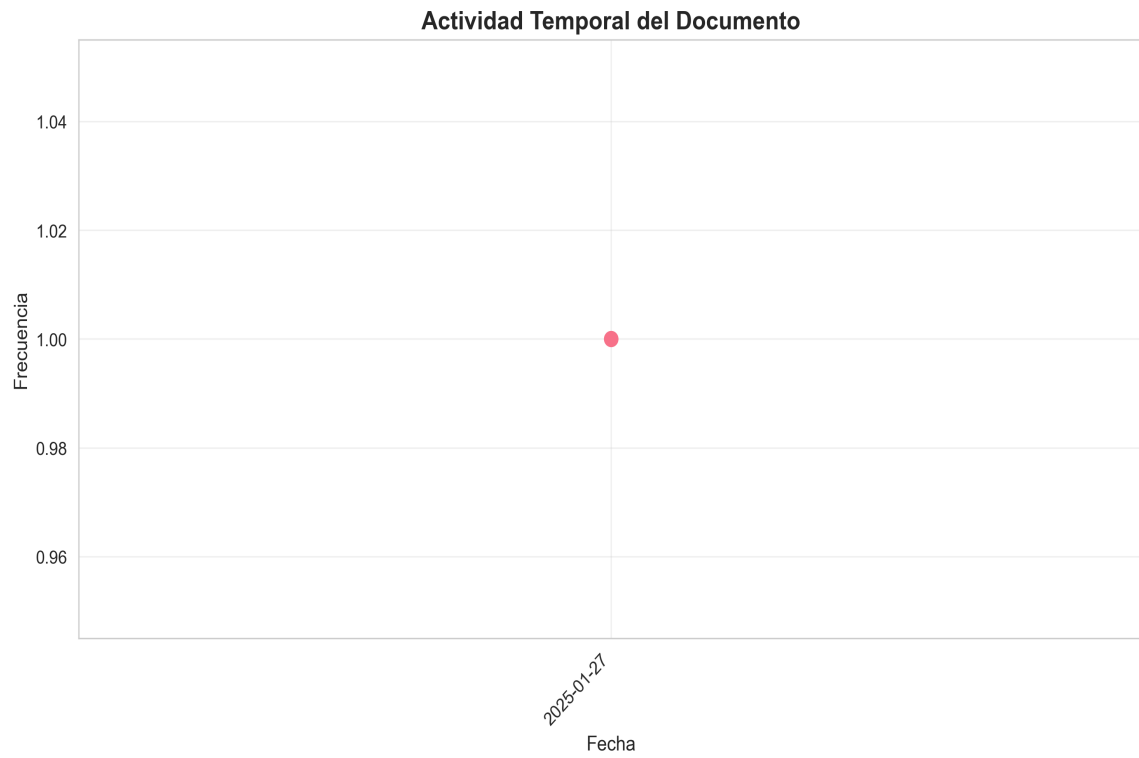


# Gráficas y Análisis





# Quick Start Guide - Production Code

*Generado el 24 de November de 2025*

# Índice de Contenido

1. Quick Start Guide - Production Code
2. Inicio Rápido
3. 1. Instalación
4. Clonar repositorio (si aplica)
5. Instalar dependencias
6. 2. Iniciar el Servidor API
7. Opción 1: Usar el script de entrada (Recomendado)
8. Opción 2: Usar uvicorn directamente
9. Opción 3: Con reload para desarrollo
10. 3. Acceder a la API
11. Uso Básico
12. Ejemplo: Almacenar Episodio en Memoria
13. Endpoint
14. Datos
15. Request
16. Ejemplo: Recuperar Episodios
17. Endpoint
18. Query
19. Request
20. Ejemplo: Usar desde Python
21. Crear aplicación
22. En un endpoint o script
23. Arquitectura Rápida
24. Capas
25. Documentación Completa
26. Configuración
27. Variables de Entorno
28. API Keys (opcional)
29. Configuración
30. Archivo de Configuración

# Quick Start Guide - Production Code

- **Versión**: 2.0.0
- **Última actualización**: 2025-01-27
- ---

## Inicio Rápido

### 1. Instalación

#### Clonar repositorio (si aplica)

```
git clone  
cd production_code
```

#### Instalar dependencias

```
pip install -r requirements.txt
```

### 2. Iniciar el Servidor API

#### Opción 1: Usar el script de entrada (Recomendado)

```
python api_server.py
```

#### Opción 2: Usar uvicorn directamente

```
uvicorn application:app --host 0.0.0.0 --port 8000
```

#### Opción 3: Con reload para desarrollo

```
python api_server.py --reload
```

### 3. Acceder a la API

- - **API**: <http://localhost:8000>
- - **Documentación**: <http://localhost:8000/docs>

- - **Dashboard**: <http://localhost:8000/dashboard>
- - **Health Check**: <http://localhost:8000/health>
- ---

## Uso Básico

### Ejemplo: Almacenar Episodio en Memoria

```
import requests
```

#### Endpoint

```
url = "http://localhost:8000/api/v1/memory/store"
```

#### Datos

```
data = {  
    "episode": [0.1, 0.2, 0.3, 0.4, 0.5],  
    "metadata": {"source": "example"},  
    "priority": 1.0  
}
```

#### Request

```
response = requests.post(url, json=data)  
print(response.json())
```

### Ejemplo: Recuperar Episodios

```
import requests
```

#### Endpoint

```
url = "http://localhost:8000/api/v1/memory/retrieve"
```

#### Query

```
data = {
```

```
"query": [0.1, 0.2, 0.3, 0.4, 0.5],  
"k": 10  
}
```

## Request

```
response = requests.post(url, json=data)  
print(response.json())
```

## Ejemplo: Usar desde Python

```
from application import create_app  
from api.dependencies import get_memory_service
```

## Crear aplicación

```
app = create_app()
```

## En un endpoint o script

```
memory_service = get_memory_service()  
result = memory_service.store_episode([0.1, 0.2, 0.3])
```

• ---

## Arquitectura Rápida

Request → API Route → Service → Domain Module → Response

## Capas

1. **Presentation** (`api/`) - Rutas y validación
2. **Application** (`services/`) - Lógica de negocio
3. **Domain** (`core/`, `memory/`, etc.) - Modelos de dominio
4. **Infrastructure** (`infrastructure/`) - Integraciones externas

• ---

## Documentación Completa

- - **README.md** - Documentación principal

- - `**`INDICE_DOCUMENTACION.md`**` - Índice completo de documentación
- - `**`RESUMEN_FINAL_MEJORAS.md`**` - Resumen de mejoras arquitectónicas
- - `**`docs/API_README.md`**` - Documentación detallada de API
- ---

## Configuración

### Variables de Entorno

#### API Keys (opcional)

```
export API_KEY=your_api_key
```

```
export OPENAI_API_KEY=your_openai_key
```

## Configuración

```
export CONFIG_PATH=config.yaml
```

### Archivo de Configuración

Crear ``config.yaml``:

```
memory:
```

```
memory_dim: 512
```

```
max_memory_size: 10000
```

```
retrieval_k: 10
```

```
redundancy:
```

```
similarity_threshold: 0.85
```

```
enable_caching: true
```

```
chat:
```

```
provider: "openai"
```

```
model: "gpt-3.5-turbo"
```

```
temperature: 0.7
```

• ---

## ■ Testing

## Ejecutar todos los tests

pytest

## Tests con cobertura

pytest --cov=. --cov-report=html

## Tests específicos

pytest tests/test\_api\_utils.py

- ---

## Troubleshooting

### Error: "Services not initialized"

- **Solución**: Asegúrate de que `initialize_services()` se llama durante el startup de la aplicación.

### Error: "Module not found"

- **Solución**: Verifica que todos los imports usan rutas de módulo correctas (ver `IMPORT_STANDARDS.md`).

### Error: "Config manager not available"

- **Solución**: Verifica que `core.config_manager` está instalado correctamente.
- ---

## Próximos Pasos

1. Leer `README.md` para documentación completa
  2. Revisar `ARCHITECTURE.md` para entender la arquitectura
  3. Consultar `docs/API_README.md` para detalles de API
  4. Ver `MEJORAS_ADICIONALES_RECOMENDADAS.md` para mejoras futuras
- ---

## Enlaces Rápidos

- **Documentación**: `INDICE_DOCUMENTACION.md`

- - **Arquitectura**: `ARCHITECTURE.md`
- - **Estándares**: `IMPORT\_STANDARDS.md`
- - **Mejoras**: `RESUMEN\_FINAL\_MEJORAS.md`
- ---
- **¿Necesitas ayuda?** Consulta la documentación completa en `INDICE\_DOCUMENTACION.md`